hexens x KATANA

Security Review Report for Katana

January 2026

Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
    - Security Review Lead
    - Scope
    - Changelog
4. Severity Structure
    - Severity characteristics
    - Issue symbolic codes
5. Findings Summary
6. Weaknesses
    - Single DVN configuration in cross-chain pathway
    - Upgrade removes pause protection for OFT transfers
    - KATCustomOFTAdapter should have approvalRequired set to false
    - Vault address not enforced in adapter initialization
    - Admin rescue function can withdraw KAT from the vault
    - Double role check in KATVault for setting or revoking LZ_BRIDGE_ROLE

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for Katana. This review included the new KAT Vault and the LayerZero OFT integration for the KAT token.

Our security assessment was a full review of the code, spanning a total of 1 week.

During our review, we did not identify any major security vulnerability.

We did identify some minor severity vulnerabilities and code optimisations.

All of our reported issues were fixed by the development team and consequently validated by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

Review Led by

Kasper Zwijsen, Head of Audits

Scope

The analyzed resources are located on:

- https://github.com/katana-network/kat-vault-lz/tree/416a2993d9524406732fc29fb27442db4fcee28c
- https://github.com/katana-network/lz-kat-upgradeable/tree/e88b7151420bae54d7ea7074abd6064ddcadba48

The issues described in this report were fixed in the following commits:

- https://github.com/katana-network/lz-kat-upgradeable/tree/f1ea9138884e849d71ad9c1be9bdabd49da7bdfd
- https://github.com/katana-network/kat-vault-lz/tree/5af6fff31bddc76e205f15227081a7ef45c06cce

Changelog

20 January 2026	Audit start
29 January 2026	Initial report
2 February 2026	Revision received
2 February 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

- **Severity Characteristics**

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium	Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.
Low	Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.
Informational	Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

- **Issue Symbolic Codes**

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity	Number of findings
Critical	0
High	0
Medium	1
Low	2
Informational	3
Total:	**6**

- Medium
- Low
- Informational

- Fixed
- Acknowledged

6. Weaknesses

This section contains the list of discovered weaknesses.

KATA1-2 | Single DVN configuration in cross-chain pathway

Fixed

Severity:	Medium	Probability:	Unlikely	Impact:	Medium

Path:

lz-kat-upgradeable/layerzero.config.ts#L37

Description:

The LayerZero OFT integration in **layerzero.config.ts** configures only a single DVN for the cross-chain pathway. The current configuration specifies **['LayerZero Labs']** as the sole required DVN with no optional DVNs.

If carried over to production without modification, it introduces a single point of failure for message verification. Should the configured DVN experience downtime, become compromised, or behave maliciously, cross-chain messages could either fail to deliver or be incorrectly validated.

LayerZero's integration guidelines(LayerZero) list as a Don't "Configure only one DVN for a pathway and treat it as production-ready." Production deployments should use more than one DVN per pathway to ensure redundancy and independent verification.

```
const pathways: TwoWayConfig[] = [
  [
    katanaContract, // Chain A contract
    bscContract, // Chain B contract
    [['LayerZero Labs'], []], // [ requiredDVN[], [ optionalDVN[], threshold ] ]
    [10, 10], // [A to B confirmations, B to A confirmations]
    [EVM_ENFORCED_OPTIONS, EVM_ENFORCED_OPTIONS], // Chain B enforcedOptions, Chain A
enforcedOptions
  ],
]
```

Remediation:

Update the pathway configuration to include at least two DVNs from independent operators before deploying to production. Configure both required and optional DVNs appropriately, and set the optional threshold to require verification from multiple DVNs before messages are considered valid. Verify that the selected DVN addresses match the official addresses published in LayerZero's documentation for the target networks.

KATA1-1 | Upgrade removes pause protection for OFT transfers

Acknowledged

Severity:	Low	Probability:	Unlikely	Impact:	Low

Path:

lz-kat-upgradeable/contracts/KATCustomOFTUpgradeable.sol:_debit#L43-L51

Description:

The original custom implementation in **KATCustomOFTUpgradeable.sol** explicitly overrides **_debit()** and enforces the **whenNotPaused** check.

However, the upgraded contract does not override **_debit()** and instead relies on the parent implementation from **OFTUpgradeable**, which does not include pause protection. Although **PausableUpgradeable** is inherited and initialized, it is not applied to the bridge execution path.

```
// KATCustomOFTUpgradeable.sol
function _debit(
    address _from,
    uint256 _amountLD,
    uint256 _minAmountLD,
    uint32 _dstEid
)
    internal
    virtual
    override
    whenNotPaused  // pause protection enforced
    returns (uint256 amountSentLD, uint256 amountReceivedLD)
{
    return super._debit(_from, _amountLD, _minAmountLD, _dstEid);
}
```

```
// KATOFTUpgradeable.sol
contract KATOFTUpgradeable is OFTUpgradeable, PausableUpgradeable {
    constructor(address _lzEndpoint) OFTUpgradeable(_lzEndpoint) {
        _disableInitializers();
    }

    function initialize(
        string memory _name,
        string memory _symbol,
        address _delegate
    ) public initializer {
        __Ownable_init(_delegate);
        __OFT_init(_name, _symbol, _delegate);
        __Pausable_init();
    }

    // _debit() is NOT overridden
    // → pause check is removed after upgrade
}
```

Remediation:

Consider overriding **_debit()** in **KATOFTUpgradeable** and reapplying the **whenNotPaused** modifier to ensure that pause protection is preserved across upgrades.

KATA1-6 | KATCustomOFTAdapter should have approvalRequired set to false

Acknowledged

Severity:	Low	Probability:	Unlikely	Impact:	Low

Path:

lz-kat-upgradeable/contracts/KATCustomOFTAdapterUpgradeable.sol:approvalRequired

Description:

The contract **KATCustomOFTAdapterUpgradeable.sol** inherits from **OFTAdapterUpgradeable**, which has a public function **approvalRequired** that returns either **true** or **false** on whether the debit requires approval or not: devtools/packages/oft-evm-upgradeable/contracts/oft/OFTAdapterUpgradeable.sol at 1ddb661d42941f7d8a342b4f2a28e9502c96bdcf · LayerZero-Labs/devtools

For the default behavior of **OFTAdapterUpgradeable**, it would require approval, but the custom adapter overwrites **_debit** and no longer requires approval.

```
/**
 * @notice Indicates whether the OFT contract requires approval of the 'token()' to send.
 * @return requiresApproval Needs approval of the underlying token implementation.
 *
 * @dev In the case of default OFTAdapter, approval is required.
 * @dev In non-default OFTAdapter contracts with something like mint and burn privileges, it would NOT need approval.
 */
function approvalRequired() external pure virtual returns (bool) {
    return true;
}
```

Remediation:

Override the **approvalRequired** function to return false.

KATA1-3 | Vault address not enforced in adapter initialization

Fixed

Severity:	Informational	Probability:	Rare	Impact:	Informational

Path:

lz-kat-upgradeable/contracts/KATCustomOFTAdapterUpgradeable.sol#L31

Description:

The adapter does not set the vault during initialization and does not validate that the configured vault is a contract. If the vault remains unset or is set to an EOA, the **transferKat** call succeeds as an empty call, so **_debit** can proceed without actually locking tokens. A **BRIDGE_USER** can therefore trigger cross-chain transfers while no tokens are moved into the vault. This is primarily a configuration risk and is observable in the adapter's initialization and vault setter paths.

```
function initialize(address _delegate) public initializer {
    __Ownable_init(_delegate);
    __AccessControl_init();
    __OFTAdapter_init(_delegate);

    _grantRole(DEFAULT_ADMIN_ROLE, _delegate);
}

function _debit(
    address,
    /*_from*/
    uint256 _amountLD,
    uint256 _minAmountLD,
    uint32 _dstEid
)
    internal
    virtual
    override
    onlyRole(BRIDGE_USER)
    returns (uint256 amountSentLD, uint256 amountReceivedLD)
{
    emit BridgeInitiated(msg.sender, _dstEid, _amountLD);
    (amountSentLD, amountReceivedLD) = _debitView(_amountLD, _minAmountLD, _dstEid);
    _getStorage().vault.transferKat(address(this), amountSentLD);
}
```

Remediation:

Require the **$.vault** variable to be set during initialization and validate that the vault address is a deployed contract. Add a guard so **_debit** reverts if the vault is unset or not a contract, and ensure the setter enforces the same constraints.

KATA1-4 | Admin rescue function can withdraw KAT from the vault

Acknowledged

Severity:	Informational	Probability:	Rare	Impact:	Informational

Path:

kat-vault-lz/src/KATVault.sol#L72

Description:

The vault includes a **rescueTokens** function that allows the admin role to transfer any ERC20 token held by the vault, including KAT. This function is callable even when the vault is paused. If the vault is intended to act as a strict lock for bridged KAT, this design permits administrative withdrawal and therefore weakens that lock assumption.

```
    function rescueTokens(address token, address to, uint256 amount) external
onlyRole(DEFAULT_ADMIN_ROLE) {
        require(token != address(0), "KATVault: token is zero address");
        require(to != address(0), "KATVault: to is zero address");
        require(amount > 0, "KATVault: amount is zero");
        IERC20(token).safeTransfer(to, amount);
        emit TokensRescued(token, to, amount);
    }
```

Remediation:

Restrict the rescue mechanism so it cannot withdraw KAT, or add safeguards that prevent administrative extraction of locked KAT under normal operation. Ensure any remaining administrative withdrawal capability aligns with the intended lock model.

We also highly recommend to have a time lock as owner, such that there is some time between publishing and execution of the admin rescue.

KATA1-5 | Double role check in KATVault for setting or revoking LZ_BRIDGE_ROLE

Fixed

Severity:	Informational	Probability:	Rare	Impact:	Informational

Path:

kat-vault-lz/src/KATVault.sol:grantLZBridgeRole, revokeLZBridgeRole#L58-L64

Description:

The KATVault is managed by the owner with the **DEFAULT_ADMIN_ROLE**, which allows for setting and revoking the **LZ_BRIDGE_ROLE** using the corresponding functions **grantLZBridgeRole** and **revokeLZBridgeRole**.

Both of these function use the modifier **onlyRole(DEFAULT_ADMIN_ROLE)** and then call into the AccessControl functions **grantRole** and **revokeRole**.

However, these functions are the public variants of OpenZeppelin's **AccessControl.sol** and already contain the modifier **onlyRole(getRoleAdmin(role))**.

By default, the role admin for any new role is the **DEFAULT_ADMIN_ROLE**, so it would be a double check for the **DEFAULT_ADMIN_ROLE** role.

```
function grantLZBridgeRole(address account) external onlyRole(DEFAULT_ADMIN_ROLE) {
    grantRole(LZ_BRIDGE_ROLE, account);
}

function revokeLZBridgeRole(address account) external onlyRole(DEFAULT_ADMIN_ROLE) {
    revokeRole(LZ_BRIDGE_ROLE, account);
}
```

Remediation:

We recommend using the internal **_grantRole** function instead. This will have the same effect, except for the double role check.

hexens x KATANA